

LAPORAN TUGAS PRAKTIKUM
“CERTAN”



Nama: Aditia Xaverius Arapenta Tarigan

NIM: 42324038

Program Studi: D3-Teknologi Informasi

INSTITUT TEKNOLOGI DEL

TAHUN AJARAN 2026/2027

FAKULTAS VOKASI

Praktikum Simple Linier Regression

```
import pandas as pd
df = pd.read_csv('Salary_Data.csv')
print(df.head())
```

✓ 0.0s

	YearsExperience	Salary
0	1.1	39343.0
1	1.3	46205.0
2	1.5	37731.0
3	2.0	43525.0
4	2.2	39891.0

Merupakan cara membaca file Salary data.csv

```
x = df.iloc[:, :-1].values
y = df.iloc[:, 1].values
```

✓ 0.0s

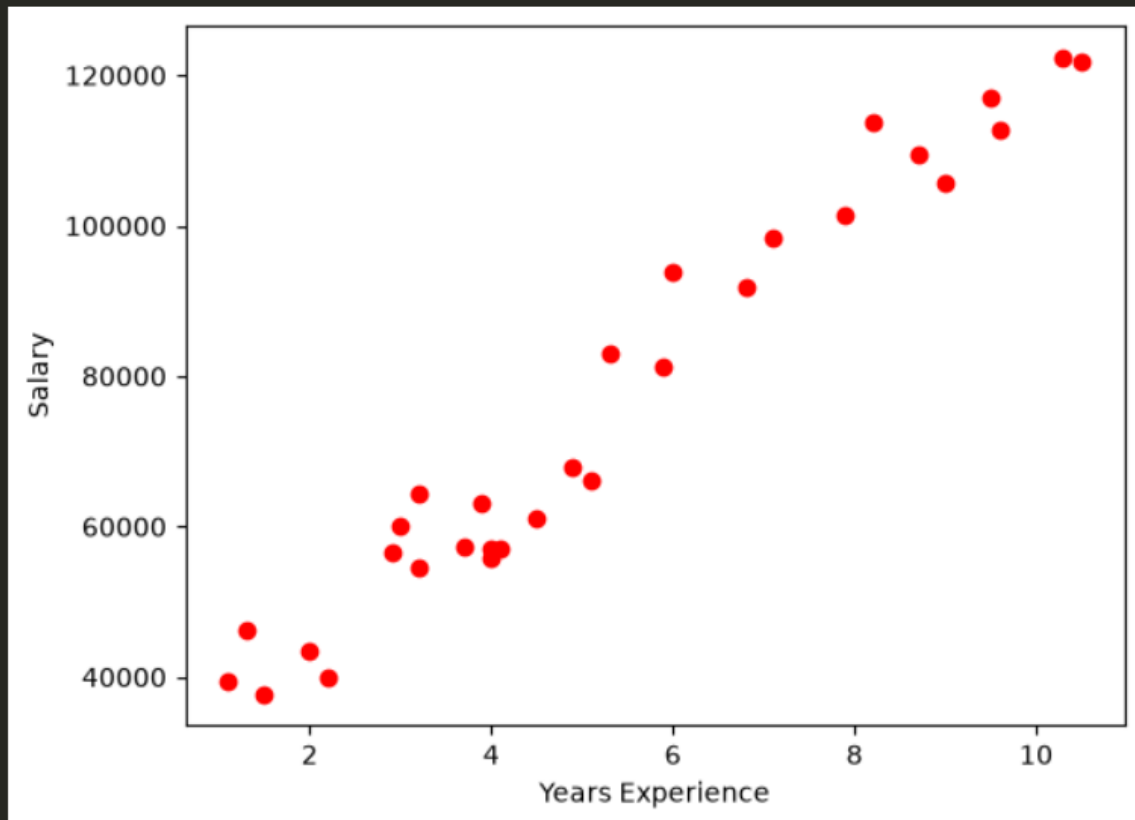
Kode tersebut digunakan untuk memisahkan data menjadi variabel input (X) dan variabel target (y), biasanya dalam proses *machine learning*.

Pada `X = df.iloc[:, :-1].values`, `df.iloc[:, :-1]` mengambil semua baris dan semua kolom kecuali kolom terakhir, sehingga kolom-kolom tersebut menjadi data yang digunakan untuk melakukan prediksi. Sedangkan

Pada `y = df.iloc[:, 1].values` mengambil semua baris pada kolom ke-2 (1 karena indeks Python dimulai dari 0) sebagai target atau hasil yang ingin diprediksi. Jadi sederhananya, X berisi data masukan dan y berisi nilai target yang akan dipelajari oleh model.

```
import matplotlib.pyplot as plt
plt.scatter(X, y, color = "red")
plt.xlabel('Years Experience')
plt.ylabel('Salary')
plt.show()
```

✓ 0.2s



Kode tersebut digunakan untuk membuat grafik scatter (titik) yang menunjukkan hubungan antara lama pengalaman kerja dan gaji. `import matplotlib.pyplot as plt` digunakan untuk memanggil library matplotlib agar bisa membuat grafik, kemudian `plt.scatter(X, y, color="red")` membuat titik-titik berwarna merah dengan X sebagai Years Experience (lama pengalaman kerja) dan y sebagai Salary (gaji). `plt.xlabel('Years Experience')` memberi nama pada sumbu X, sedangkan `plt.ylabel('Salary')` memberi nama pada sumbu Y. Terakhir, `plt.show()` digunakan untuk menampilkan grafik. Dari grafik tersebut dapat dilihat bahwa secara umum semakin banyak tahun pengalaman kerja, semakin tinggi gaji, karena titik-titik cenderung bergerak naik dari kiri bawah ke kanan atas.

```
#bagi dataset jadi train dan test 80:20
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_state = 0)
✓ 0.0s
```

Kode pada gambar tersebut berfungsi untuk membagi secara acak keseluruhan data Anda menjadi dua kelompok, yaitu data untuk "belajar" (*training*) dan data untuk "ujian" (*testing*). Dengan mengatur `test_size = 0.2`, Anda menugaskan komputer untuk menggunakan 80% data sebagai bahan melatih model *machine learning* (`x_train` dan `y_train`), lalu menyisihkan 20% sisanya sebagai soal ujian (`x_test` dan `y_test`) untuk mengukur seberapa akurat tebakan model tersebut pada data yang belum pernah dilihat sebelumnya. Ibarat proses belajar di sekolah, 80% soal digunakan untuk latihan harian agar komputer memahami pola, sedangkan 20% sisanya disimpan khusus untuk ujian akhir demi memastikan komputer benar-benar paham dan bukan sekadar menghafal. Selain itu, parameter `random_state = 0` digunakan agar pengacakan data ini selalu menghasilkan susunan yang sama persis setiap kali kode dijalankan ulang, sehingga eksperimen Anda tetap konsisten.

```
# Fitting Simple Linear Regression to the Training set
from sklearn.linear_model import LinearRegression
regressor = LinearRegression()
regressor.fit(X_train, y_train)
✓ 0.0s
```

▼ LinearRegression ⓘ ?

- Parameters
- Fitted attributes

Kode tersebut digunakan untuk membuat dan melatih model Simple Linear Regression (regresi linear sederhana). `from sklearn.linear_model import LinearRegression` digunakan untuk mengambil fungsi regresi linear dari library Scikit-learn, kemudian `regressor = LinearRegression()` membuat sebuah objek model regresi. Setelah itu, `regressor.fit(X_train, y_train)` digunakan untuk melatih model menggunakan data training, yaitu `X_train` sebagai data pengalaman kerja dan `y_train` sebagai data gaji. Setelah proses ini selesai, model sudah mempelajari hubungan antara pengalaman kerja dan gaji sehingga nantinya dapat digunakan untuk memprediksi gaji berdasarkan pengalaman kerja.

```

a = regressor.intercept_
b = regressor.coef_[0]
print("Estimated model intercept, a:", a)
print("Estimated model slope, b:", b)
print(f" $\hat{y} = \{a:.2f\} + \{b:.2f\}x$ ")

```

4] ✓ 0.0s

```

Estimated model intercept, a: 26780.09915062818
Estimated model slope, b: 9312.575126729189
 $\hat{y} = 26780.10 + 9312.58x$ 

```

Kode pada gambar tersebut berfungsi untuk mengambil hasil perhitungan matematis dari model regresi linear yang telah dilatih, lalu merangkainya menjadi sebuah rumus prediksi. Variabel a digunakan untuk mengambil nilai *intercept* atau titik awal, yang dalam konteks data sebelumnya mengibaratkan taksiran gaji dasar seseorang (sekitar 26.780) ketika ia belum memiliki pengalaman kerja sama sekali. Sementara itu, variabel b mengambil nilai *slope* atau tingkat kemiringan, yang menunjukkan seberapa besar taksiran gaji tersebut akan bertambah (sekitar 9.312) untuk setiap tambahan satu tahun pengalaman kerja. Pada bagian akhir, kode akan mencetak nilai-nilai tersebut ke layar dan menggabungkannya ke dalam format persamaan garis lurus $\hat{y} = 26780.10 + 9312.58x$, sehingga Anda bisa memprediksi estimasi gaji ($\hat{y}$) hanya dengan memasukkan angka jumlah tahun pengalaman kerja (x) ke dalam rumus tersebut.

```

# memprediksi hasil pada data uji (Test set) menggunakan model regresi yang telah
y_pred = regressor.predict(X_test)
y_pred

```

5] ✓ 0.0s

```

array([ 40748.96184072, 122699.62295594,  64961.65717022,  63099.14214487,
        115249.56285456, 107799.50275317])

```

Kode tersebut digunakan untuk memprediksi nilai gaji pada data uji (X_{test}) menggunakan model regresi linear yang sebelumnya sudah dilatih. `regressor.predict( $X_{\text{test}}$ )` meminta model untuk menghitung prediksi berdasarkan data pengalaman kerja yang ada di X_{test} , kemudian hasil prediksi disimpan ke dalam variabel `y_pred`. Ketika `y_pred` ditampilkan, muncul beberapa nilai seperti 40748.96, 122699.62, 64961.66, dan seterusnya, yang merupakan perkiraan gaji untuk masing-masing data pada data uji. Jadi, sederhananya, kode ini digunakan untuk mengetahui hasil prediksi model terhadap data yang belum digunakan saat proses pelatihan.

```
# Menghitung nilai R-Square, yang mengukur seberapa baik model regresi linear
# Semakin mendekati 1, semakin baik model
from sklearn.metrics import r2_score
score=r2_score(y_test,y_pred)
score
```

6]

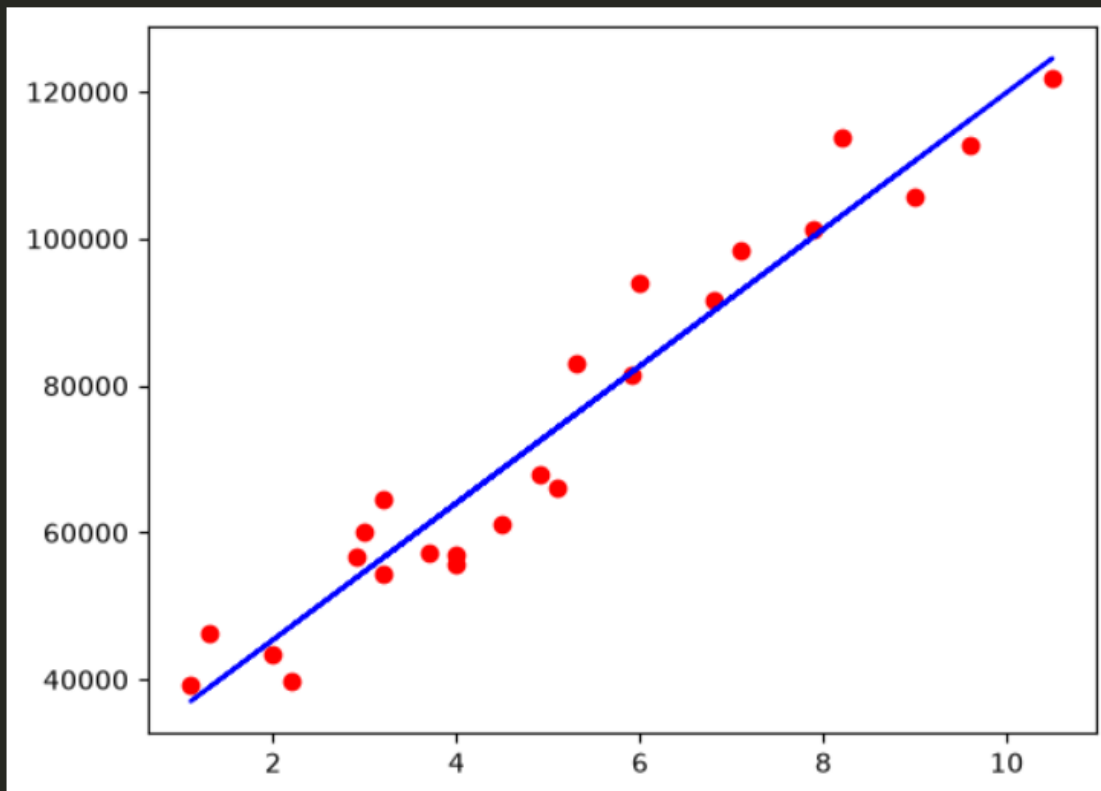
```
0.988169515729126
```

Kode tersebut digunakan untuk mengukur seberapa baik model regresi linear dalam melakukan prediksi menggunakan nilai R-Square (R^2). `r2_score(y_test, y_pred)` membandingkan nilai gaji sebenarnya (`y_test`) dengan nilai gaji hasil prediksi (`y_pred`), kemudian menghasilkan nilai R^2 sebesar 0.9881695. Nilai R^2 berada antara 0 sampai 1, dan semakin mendekati 1 berarti model semakin mampu menjelaskan hubungan pada data. Jadi, pada hasil ini nilai $0.988 \approx 98,8\%$, yang menunjukkan bahwa model dapat menjelaskan sekitar 98,8% variasi nilai gaji pada data uji, sedangkan sisanya sekitar 1,2% tidak dijelaskan oleh model.

```
# Visualizing the Training set results
plt.scatter(X_train, y_train, color = 'red')
plt.plot(X_train, regressor.predict(X_train), color = 'blue')
```

]

[<matplotlib.lines.Line2D at 0xb2ccea270>]



Kode tersebut digunakan untuk memvisualisasikan hasil regresi linear pada data training. `plt.scatter(X_train, y_train, color='red')` menampilkan titik-titik merah yang merupakan data pengalaman kerja dan gaji sebenarnya, sedangkan `plt.plot(X_train, regressor.predict(X_train), color='blue')` membuat garis biru yang menunjukkan hasil prediksi model regresi linear. Jadi, grafik ini digunakan untuk melihat seberapa baik garis regresi mengikuti pola data training. Pada gambar terlihat garis biru mengikuti kecenderungan titik-titik merah dari kiri bawah ke kanan atas, sehingga menunjukkan adanya hubungan positif antara pengalaman kerja dan gaji.

```
# Visualizing the Test set results
plt.scatter(X_test, y_test, color = 'red')
plt.plot(X_train, regressor.predict(X_train), color = 'blue')
plt.title('Salary vs Experience (Test set)')
plt.xlabel('Years of Experience')
plt.ylabel('Salary')
plt.show()
```



Kode tersebut digunakan untuk menampilkan hasil regresi linear pada data uji (test set). `plt.scatter(X_test, y_test, color='red')` menampilkan titik merah yang berisi data pengalaman kerja dan gaji sebenarnya, sedangkan `plt.plot(X_train, regressor.predict(X_train), color='blue')` menampilkan garis regresi yang dibuat dari data training sebagai acuan untuk melihat hasil prediksi pada data uji. `plt.title()` memberikan judul grafik, `plt.xlabel()` memberi nama sumbu X yaitu pengalaman kerja, `plt.ylabel()` memberi nama sumbu Y yaitu gaji, dan `plt.show()` menampilkan grafik. Dari grafik terlihat sebagian besar titik data uji berada dekat dengan garis biru, sehingga hasil prediksi model cukup dekat dengan nilai gaji sebenarnya pada data uji.

Praktikum Multiple Linier Regression

Multiple Linear Regression

```
df2 = pd.read_csv('multiple_linear_regression_dataset.csv')
print(df2.head())
```

✓ 0.0s

	age	experience	income
0	25	1	30450
1	30	3	35670
2	47	2	31580
3	32	5	40130
4	43	10	47830

Kode tersebut digunakan untuk membaca dan melihat isi awal dataset yang akan digunakan dalam Multiple Linear Regression (regresi linear berganda).

`pd.read_csv('multiple_linear_regression_dataset.csv')` digunakan untuk membaca file CSV dan menyimpannya ke dalam variabel `df2`, sedangkan `df2.head()` digunakan untuk menampilkan 5 baris pertama dari dataset. Pada data tersebut terdapat tiga kolom, yaitu `age` (usia), `experience` (pengalaman kerja), dan `income` (pendapatan). Nantinya, `age` dan `experience` dapat digunakan sebagai variabel input (X), sedangkan `income` menjadi variabel target (y) yang ingin diprediksi. Jadi, kode ini merupakan tahap awal untuk menyiapkan dan melihat data sebelum membuat model regresi linear berganda.

```
X2 = df2.iloc[:, :-1]
y2 = df2.iloc[:, 2]
```

Kode tersebut digunakan untuk memisahkan data menjadi variabel input (X2) dan variabel target (y2) pada Multiple Linear Regression. `X2 = df2.iloc[:, :-1]` mengambil semua baris dan semua kolom kecuali kolom terakhir, sehingga X2 berisi kolom `age` dan `experience` yang akan digunakan sebagai faktor untuk melakukan prediksi. Sedangkan `y2 = df2.iloc[:, 2]` mengambil semua baris pada kolom ke-3, yaitu kolom `income`, sebagai nilai target yang ingin diprediksi. Jadi, sederhananya X2 = usia dan pengalaman kerja sebagai input, sedangkan y2 = pendapatan sebagai hasil yang ingin diprediksi.

```
# Bagi dataset menjadi train dan test 80:20
X_train2, X_test2, y_train2, y_test2 = train_test_split(X2, y2, test_size = 0.2, random_state = 0)
```

Kode tersebut digunakan untuk membagi dataset menjadi data training dan data testing dengan perbandingan 80:20. Fungsi `train_test_split(X2, y2, test_size=0.2, random_state=0)` membagi X2 (data age dan experience) serta y2 (data income) menjadi empat bagian, yaitu X_train2 dan y_train2 untuk melatih model sebesar 80% data, sedangkan X_test2 dan y_test2 untuk menguji model sebesar 20% data. `random_state=0` digunakan agar pembagian data tetap sama setiap kali kode dijalankan, sehingga hasilnya dapat direproduksi.

```
regressor = LinearRegression()  
regressor.fit(X_train2, y_train2)
```

▼ LinearRegression ⓘ ?

- Parameters
- Fitted attributes

Kode tersebut digunakan untuk membuat dan melatih model Multiple Linear Regression (regresi linear berganda) menggunakan data training. `regressor = LinearRegression()` membuat objek model regresi linear, kemudian `regressor.fit(X_train2, y_train2)` melatih model menggunakan X_train2 yang berisi usia (age) dan pengalaman kerja (experience) sebagai variabel input, serta y_train2 yang berisi income sebagai target yang ingin diprediksi. Setelah proses `fit()` selesai, model sudah mempelajari hubungan antara usia, pengalaman kerja, dan pendapatan, sehingga nantinya dapat digunakan untuk memprediksi income berdasarkan usia dan pengalaman kerja.

```
# Predicting the Test set results  
y_pred2 = regressor.predict(X_test2)  
print(y_pred2)
```

```
[46487.25542968 34623.71312375 49379.11875301 38349.57565764]
```

Kode tersebut digunakan untuk memprediksi nilai income pada data testing menggunakan model Multiple Linear Regression yang sudah dilatih sebelumnya. `regressor.predict(X_test2)` menggunakan data usia (age) dan pengalaman kerja (experience) dari X_test2 untuk menghitung perkiraan pendapatan, kemudian hasilnya disimpan dalam variabel y_pred2. `print(y_pred2)` menampilkan hasil prediksi tersebut, yaitu 46487.26, 34623.71, 49379.12, dan 38349.58. Jadi, sederhananya kode ini digunakan untuk mengetahui perkiraan pendapatan berdasarkan usia dan pengalaman kerja pada data yang belum digunakan saat melatih model.

```

a_ = regressor.intercept_
b1 = regressor.coef_[0]
b2 = regressor.coef_[1]

print("Estimated model intercept, a:", a_)
print("Estimated model slope, b1:", b1)
print("Estimated model slope, b2:", b2)
print(f"ŷ = {a_:.2f} {b1:.2f}x1 + {b2:.2f}x2")

```

```

Estimated model intercept, a: 30945.793141906877
Estimated model slope, b1: -97.99352917934176
Estimated model slope, b2: 2205.90861907387
ŷ = 26780.10 -97.99x1 + 2205.91x2

```

Kode tersebut digunakan untuk mengambil nilai koefisien dari model Multiple Linear Regression dan membentuk persamaan regresinya. `regressor.intercept_` mengambil nilai intercept (a), sedangkan `regressor.coef_[0]` dan `regressor.coef_[1]` mengambil koefisien untuk variabel pertama dan kedua, yaitu age (x1) dan experience (x2). Berdasarkan hasilnya, model memiliki persamaan $\hat{y} = 26780.10 - 97.99x_1 + 2205.91x_2$, yang berarti setiap kenaikan 1 tahun usia diperkirakan mengubah income sebesar -97,99, sementara setiap tambahan 1 tahun pengalaman kerja diperkirakan meningkatkan income sebesar 2.205,91, dengan variabel lainnya dianggap tetap. Jadi, kode ini digunakan untuk mengetahui bentuk persamaan matematika yang dipelajari oleh model regresi berganda.

```

# Menghitung nilai R-Square, yang mengukur seberapa baik model regresi linear
# Semakin mendekati 1, semakin baik model
score2=r2_score(y_test2,y_pred2)
score2

```

```
0.8872690560355563
```

Kode pada gambar tersebut bertujuan untuk mengukur tingkat akurasi atau seberapa hebat model prediksi Anda bekerja dengan menghitung nilai evaluasi bernama *R-Square*. Lewat perintah `r2_score(y_test2, y_pred2)`, komputer membandingkan kumpulan data ujian yang asli (`y_test2`) dengan hasil tebakan yang dibuat oleh model (`y_pred2`). Seperti yang dijelaskan pada baris komentar, nilai *R-Square* ini ibarat nilai rapor yang berpatokan pada angka maksimal 1; semakin hasilnya mendekati angka 1, maka semakin akurat dan bagus kinerja model tersebut. Hasil akhir yang muncul sebesar 0.887 (atau sekitar 88,7%) menunjukkan bahwa model Anda sudah bekerja dengan sangat baik dan mampu menebak pola data dengan tingkat presisi yang tinggi.

Tugas

1. Simple Linear Regression: prediksi score (y) berdasarkan waktu belajar (X) dari dataset <https://www.kaggle.com/datasets/snmahsa/student-score-suitable-forlinear-regression>

```
print("Nama:aditia tarigan")
print("NIM:42324038")
print()
import pandas as pd
df = pd.read_csv("rounded_hours_student_scores.csv")
print(df.head())
```

0.0s

Nama:aditia tarigan
NIM:42324038

	Hours	Scores
0	1.1	41
1	1.2	40
2	1.4	38
3	1.5	39
4	1.6	36

Merupakan cara untuk membaca isi dari si file rounded_hours

```
X = df.iloc[:, :-1].values
y = df.iloc[:, 1].values
```

Kode tersebut digunakan untuk memisahkan data menjadi variabel input (X) dan variabel target (y), biasanya dalam proses *machine learning*.

Pada `X = df.iloc[:, :-1].values`, `df.iloc[:, :-1]` mengambil semua baris dan semua kolom kecuali kolom terakhir, sehingga kolom-kolom tersebut menjadi data yang digunakan untuk melakukan prediksi. Sedangkan

Pada `y = df.iloc[:, 1].values` mengambil semua baris pada kolom ke-2 (1 karena indeks Python dimulai dari 0) sebagai target atau hasil yang ingin diprediksi. Jadi sederhana, X berisi data masukan dan y berisi nilai target yang akan dipelajari oleh model.

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)
```

Kode pada gambar tersebut merupakan perintah yang bertugas memecah keseluruhan data Anda (variabel X dan y) menjadi dua kelompok terpisah untuk keperluan *machine learning*. Dengan mengatur nilai `test_size=0.2`, Anda menginstruksikan komputer untuk menggunakan 80% dari total data sebagai bahan materi belajar untuk melatih model (`X_train` dan `y_train`), lalu menyimpan 20% sisanya secara khusus sebagai soal ujian akhir (`X_test` dan `y_test`) untuk mengukur seberapa akurat hasil pembelajaran model tersebut. Sementara itu, parameter `random_state=0` dipasang untuk mengunci pola pengacakan data, sehingga setiap kali kode ini dijalankan ulang, susunan data yang terbagi akan selalu sama persis dan menjaga hasil eksperimen Anda tetap konsisten.

```
regressor = LinearRegression()
regressor.fit(X_train, y_train)
```

✓ 0.0s

LinearRegression ⓘ ?

- Parameters
- Fitted attributes

Kode tersebut digunakan untuk membuat dan melatih model Multiple Linear Regression (regresi linear berganda) menggunakan data training. `regressor = LinearRegression()` membuat objek model regresi linear, kemudian `regressor.fit(X_train2, y_train2)` melatih model menggunakan `X_train2` yang berisi usia (age) dan pengalaman kerja (experience) sebagai variabel input, serta `y_train2` yang berisi income sebagai target yang ingin diprediksi. Setelah proses `fit()` selesai, model sudah mempelajari hubungan antara usia, pengalaman kerja, dan pendapatan, sehingga nantinya dapat digunakan untuk memprediksi income berdasarkan usia dan pengalaman kerja.

```
# Memprediksi hasil pada data uji
y_pred = regressor.predict(X_test)

# Mengambil nilai intercept (a) dan slope (b)
a = regressor.intercept_
b = regressor.coef_[0]

print("Estimated model intercept, a:", a)
print("Estimated model slope, b:", b)
print(f"Persamaan: y = {a:.2f} + {b:.2f}x")
```

✓ 0.0s

```
Estimated model intercept, a: 37.89028997006414
Estimated model slope, b: 1.6690585851433408
Persamaan: y = 37.89 + 1.67x
```

Kode pada gambar tersebut berfungsi untuk melakukan prediksi akhir menggunakan model *machine learning* yang sudah dilatih sekaligus merangkumnya menjadi sebuah rumus matematika. Pada baris awal, perintah `regressor.predict(X_test)` bertugas menyuruh model untuk menghasilkan tebakan (`y_pred`) berdasarkan sekumpulan data ujian yang belum pernah ia lihat sebelumnya. Selanjutnya, kode mengambil dua komponen utama dari model regresi tersebut, yaitu nilai *intercept* (a) sebagai titik awal yang bernilai sekitar 37,89, dan nilai *slope* (b) sebagai tingkat kemiringan yang bernilai sekitar 1,67. Terakhir, kode mencetak nilai-nilai ini ke layar dan menggabungkannya ke dalam format persamaan garis lurus $y = 37.89 + 1.67x$, yang berarti Anda dapat memperkirakan nilai output (y) dengan cara mengalikan data input Anda (x) dengan 1,67 dan menambahkannya dengan basis awal 37,89.

```
score = r2_score(y_test, y_pred)
print("R-Square score:", score)
```

✓ 0.0s

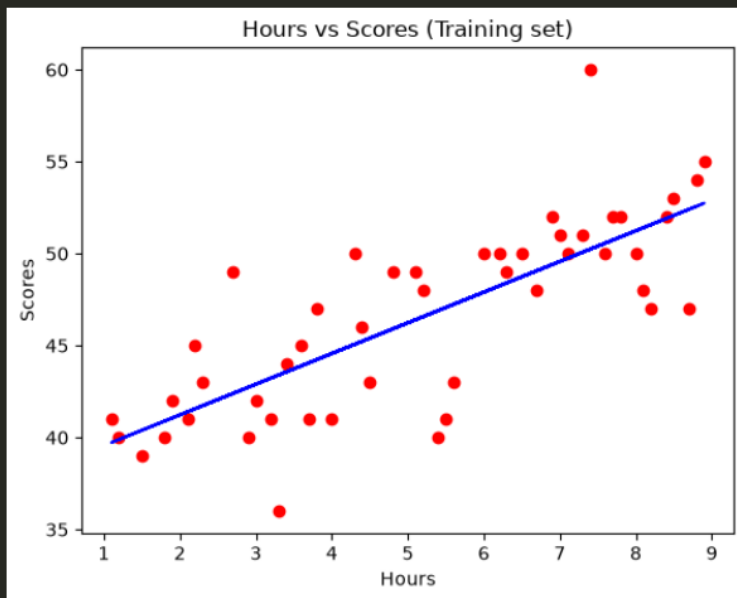
R-Square score: 0.6886702273580019

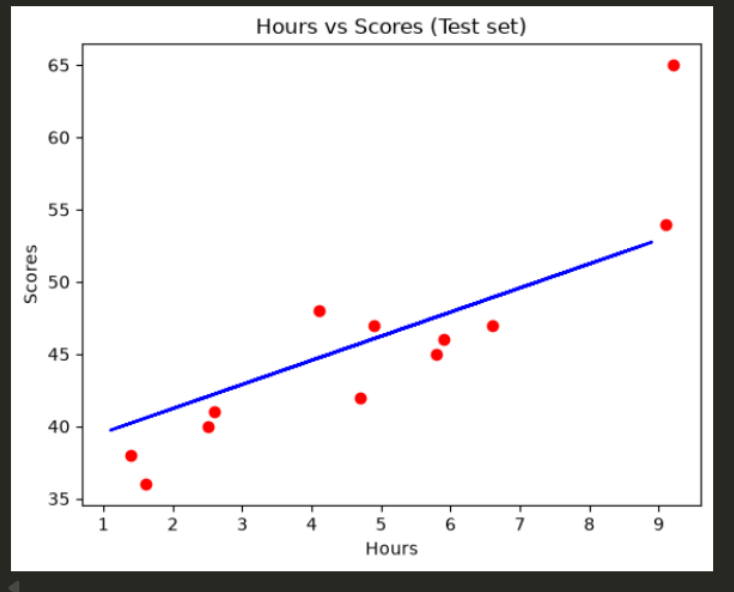
Kode pada gambar tersebut berfungsi untuk menilai seberapa akurat atau pintar model *machine learning* Anda dalam membuat tebakan dengan cara menghitung metrik evaluasi yang disebut skor *R-Square*. Melalui perintah `r2_score(y_test, y_pred)`, komputer diibaratkan sedang mengoreksi "kertas ujian" dengan membandingkan kunci jawaban asli dari data pengujian (`y_test`) dengan hasil prediksi atau tebakan yang telah dibuat oleh model (`y_pred`). Skor *R-Square* memiliki nilai maksimal 1 (atau 100%), sehingga hasil keluaran sebesar 0.688 menunjukkan bahwa model Anda sudah memiliki performa yang cukup baik, di mana ia berhasil memahami dan memprediksi sekitar 68,8% pola dari keseluruhan data yang diberikan kepadanya.

```
# Visualisasi Training Set
plt.scatter(X_train, y_train, color='red')
plt.plot(X_train, regressor.predict(X_train), color='blue')
plt.title('Hours vs Scores (Training set)')
plt.xlabel('Hours')
plt.ylabel('Scores')
plt.show()

# Visualisasi Test Set
plt.scatter(X_test, y_test, color='red')
plt.plot(X_train, regressor.predict(X_train), color='blue')
plt.title('Hours vs Scores (Test set)')
plt.xlabel('Hours')
plt.ylabel('Scores')
plt.show()
```

✓ 0.6s





Kode dan grafik pada gambar tersebut berfungsi untuk memvisualisasikan seberapa baik model *machine learning* Anda dalam menangkap pola dari data, dengan cara menggambarannya ke dalam bentuk plot. Titik-titik berwarna merah merepresentasikan sebaran data aktual yang menghubungkan lama waktu belajar (*Hours*) dengan nilai yang didapat (*Scores*), sedangkan garis lurus tebal berwarna biru adalah garis prediksi (regresi) atau tren utama yang berhasil ditarik oleh model berdasarkan pola titik-titik tersebut. Melalui visualisasi ini, Anda bisa melihat secara langsung seberapa pas tebakan model (garis biru) dalam mewakili atau mendekati data aslinya (titik merah), yang pada kode tersebut telah diatur agar dapat ditampilkan secara bergantian baik untuk kelompok data yang digunakan saat latihan (*Training set*) maupun data saat ujian (*Test set*).

2. Multiple Linear Regression: prediksi sales (y) berdasarkan data marketing (x) di youtube, facebook, newspaper, dari dataset <https://www.kaggle.com/datasets/fayejavad/marketing-linear-multiple-regression>

2. Multiple Linear Regression: prediksi sales (y) berdasarkan data marketing (x) di youtube, facebook, newspaper, dari dataset <https://www.kaggle.com/datasets/fayejavad/marketing-linear-multiple-regression>

```
import pandas as pd
data_mkt = pd.read_csv('Marketing_Data.csv')
print(data_mkt.head())
```

✓ 0.0s

	youtube	facebook	newspaper	sales
0	84.72	19.20	48.96	12.60
1	351.48	33.96	51.84	25.68
2	135.48	20.88	46.32	14.28
3	116.64	1.80	36.00	11.52
4	318.72	24.00	0.36	20.88

Merupakan cara mengimport dan membaca file dari Marketing_Data.csv

```
X_mkt = data_mkt.iloc[:, :-1]
y_mkt = data_mkt.iloc[:, 3]
```

✓ 0.0s

Kode pada gambar tersebut berfungsi untuk memisahkan sebuah tabel data utama bernama `data_mkt` menjadi dua kelompok yang berbeda, yaitu kelompok data input (fitur) dan kelompok data output (target prediksi). Pada baris pertama, perintah `iloc[:, :-1]`

menginstruksikan komputer untuk mengambil seluruh baris data beserta semua kolom *kecuali* kolom yang paling terakhir, lalu menyimpannya ke dalam variabel `X_mkt` sebagai kumpulan informasi atau petunjuk yang akan dipelajari oleh model. Sementara itu pada baris kedua, perintah `iloc[:, 3]` secara spesifik hanya mengambil semua baris dari kolom keempat (karena indeks komputer selalu dihitung mulai dari 0, sehingga indeks 3 berarti kolom ke-4) untuk disimpan sebagai variabel `y_mkt`. Ibarat mempersiapkan buku latihan soal, Anda sedang memisahkan kumpulan petunjuk cerita ke dalam keranjang `X_mkt`, dan secara khusus menyisihkan kolom yang berisi kunci jawaban akhir ke dalam keranjang `y_mkt` agar komputer bisa belajar membedakan serta mencari pola di antara keduanya.

```
X_train_mkt, X_test_mkt, y_train_mkt, y_test_mkt = train_test_split(X_mkt, y_mkt, test_size=0.2, random_state=0)
```

✓ 0.0s

Kode pada gambar tersebut berfungsi untuk memecah kumpulan data Anda (variabel `X_mkt` dan `y_mkt`) menjadi dua bagian terpisah, yaitu sekumpulan data untuk melatih model (*training*) dan sekumpulan data untuk mengujinya (*testing*). Dengan menetapkan `test_size=0.2`, Anda menginstruksikan komputer untuk mengambil 80% dari total data sebagai bahan belajar harian (`X_train_mkt` dan `y_train_mkt`), lalu mengamankan 20% sisanya secara khusus sebagai soal ujian akhir (`X_test_mkt` dan `y_test_mkt`) untuk mengukur seberapa akurat hasil pembelajaran model tersebut. Selain itu, pengaturan `random_state=0` bertindak sebagai pengunci pola pengacakan, yang memastikan bahwa setiap kali kode ini dijalankan ulang, susunan data yang terbagi akan selalu sama persis agar hasil eksperimen Anda tetap konsisten dan tidak berubah-ubah.

```
regressor_mkt = LinearRegression()
regressor_mkt.fit(X_train_mkt, y_train_mkt)
```

✓ 0.1s

▼ LinearRegression ⓘ ?

- Parameters
- Fitted attributes

Kode pada gambar tersebut merupakan tahapan utama dalam *machine learning* di mana Anda menciptakan sekaligus melatih sebuah model kecerdasan buatan. Pada baris pertama, perintah `LinearRegression()` berfungsi untuk menyiapkan sebuah model prediksi berbasis regresi linear yang masih kosong (ibarat seorang "murid" baru yang siap belajar), lalu menyimpannya ke dalam variabel bernama `regressor_mkt`. Selanjutnya pada baris kedua, perintah `.fit(X_train_mkt, y_train_mkt)` bertindak sebagai proses belajar itu sendiri; Anda menginstruksikan model tersebut untuk mempelajari pola hubungan antara sekumpulan soal latihan (`X_train_mkt`) dan kunci jawabannya (`y_train_mkt`). Melalui tahapan *fitting* inilah komputer menyesuaikan perhitungannya secara otomatis demi menemukan rumus terbaik, agar nantinya ia mampu membuat tebakan yang akurat ketika dihadapkan pada data-data baru.

```

y_pred_mkt = regressor_mkt.predict(X_test_mkt)
print("Hasil Prediksi Sales:")
print(y_pred_mkt)

```

✓ 0.0s

Hasil Prediksi Sales:

```

[17.47637032 16.37438329 23.03375348 24.76366742 18.44000541  9.88587684
 27.58148165 28.87880859 23.06393669 19.0636305  13.21803757 14.34999561
  9.23890874 13.14124811 10.24291776 16.2522827  20.0147228 25.73237296
 12.05155149 13.89264368 19.7192218  10.95264453 22.23454235 27.70965086
 20.47231834 11.94972356 19.4677982  15.98774074 23.2733346  20.3406546
 25.5594432  18.45251872  6.66383729 20.57898229 18.94707082]

```

Kode pada gambar tersebut berfungsi untuk menugaskan model *machine learning* yang sebelumnya telah dilatih agar mulai membuat tebakan nyata menggunakan data ujian. Melalui perintah `regressor_mkt.predict(X_test_mkt)`, Anda seolah-olah sedang memberikan sekumpulan "soal ujian" baru (`X_test_mkt`) kepada model, yang kemudian dihitung dan dijawab oleh model untuk menghasilkan estimasi nilai penjualan (*sales*). Jawaban atau hasil prediksi dari komputer ini kemudian ditampung ke dalam sebuah variabel bernama `y_pred_mkt`. Setelah perhitungan selesai, perintah `print` bertugas untuk menampilkan teks "Hasil Prediksi Sales:" beserta seluruh deretan angka hasil tebakan tersebut ke layar, sehingga Anda bisa langsung melihat secara praktis berapa perkiraan angka penjualan untuk setiap data yang sedang diuji.

```

a_mkt = regressor_mkt.intercept_
b1_mkt = regressor_mkt.coef_[0]
b2_mkt = regressor_mkt.coef_[1]
b3_mkt = regressor_mkt.coef_[2]

print("Estimated model intercept, a:", a_mkt)
print("Estimated model slope, b1:", b1_mkt)
print("Estimated model slope, b2:", b2_mkt)
print("Estimated model slope, b3:", b3_mkt)
print(f"y = {a_mkt:.2f} + {b1_mkt:.2f}x1 + {b2_mkt:.2f}x2 + {b3_mkt:.2f}x3")

```

81 ✓ 0.0s

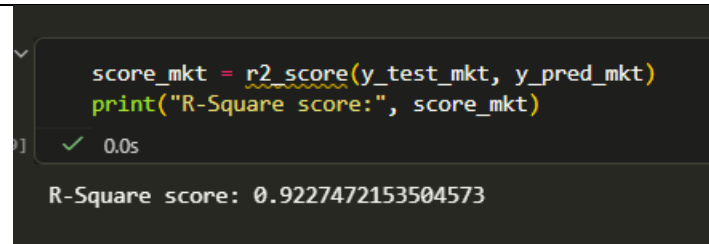
```

Estimated model intercept, a: 3.855843431588168
Estimated model slope, b1: 0.043812661462685164
Estimated model slope, b2: 0.18387483267122112
Estimated model slope, b3: 0.004809634069435298
y = 3.86 + 0.04x1 + 0.18x2 + 0.00x3

```

Kode pada gambar tersebut berfungsi untuk membongkar hasil perhitungan dari model *machine learning* Anda dan merangkainya menjadi sebuah rumus matematika lengkap untuk prediksi yang melibatkan lebih dari satu faktor (disebut regresi linear berganda). Variabel `a_mkt` mengambil nilai *intercept* atau nilai dasar sebesar 3,86, yang ibaratnya merupakan perkiraan hasil awal jika semua faktor input tidak ada atau bernilai nol. Selanjutnya, variabel `b1_mkt`, `b2_mkt`, dan `b3_mkt` masing-masing mengambil nilai *slope* (kemiringan atau bobot) yang menunjukkan seberapa besar tiap-tiap faktor input yang berbeda (faktor ke-1, ke-2, dan ke-3) akan mempengaruhi hasil akhir. Di bagian akhir, kode tersebut mencetak angka-angka

ini ke layar dan menggabungkannya ke dalam format persamaan $y = 3.86 + 0.04x_1 + 0.18x_2 + 0.00x_3$, sehingga Anda bisa menghitung estimasi target akhir (y) dengan mudah hanya dengan memasukkan data Anda ke dalam ketiga variabel x_1 , x_2 , dan x_3 tersebut secara bersamaan.

A screenshot of a Jupyter Notebook cell. The code inside the cell is:

```
score_mkt = r2_score(y_test_mkt, y_pred_mkt)
print("R-Square score:", score_mkt)
```

 Below the code, there is a green checkmark icon and the text "0.0s". At the bottom of the cell, the output is displayed: "R-Square score: 0.9227472153504573".

```
score_mkt = r2_score(y_test_mkt, y_pred_mkt)
print("R-Square score:", score_mkt)
```

✓ 0.0s

R-Square score: 0.9227472153504573

Kode pada gambar tersebut berfungsi untuk mengevaluasi seberapa akurat kinerja model *machine learning* Anda dengan cara menghitung metrik yang dinamakan skor *R-Square*. Melalui perintah `r2_score(y_test_mkt, y_pred_mkt)`, komputer mengibaratkan dirinya sebagai pengoreksi ujian yang sedang membandingkan data aktual atau kunci jawaban asli (`y_test_mkt`) dengan hasil prediksi tebakan dari model Anda (`y_pred_mkt`). Karena nilai sempurna untuk skor *R-Square* adalah 1 (yang setara dengan keakuratan 100%), hasil cetak di layar yang menunjukkan angka sebesar 0.922 (atau sekitar 92,2%) mengartikan bahwa model prediksi Anda sudah berkinerja dengan sangat cemerlang dan memiliki tingkat keakuratan yang amat tinggi dalam mengenali serta memprediksi pola data tersebut.